

Local graph complexity as a predictor of Giraffe indexing failure and mapping accuracy in a human pangenome graph

Kazumichi Fujiwara¹ and Yosuke Kawai¹

¹ National Institute of Genetics

BioHackathon series:

DBCLS BioHackathon 2026

Matsuyama, Japan, 2026

BH26

Submitted: 18 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC-BY).

Published by [BioHackrXiv.org](https://biohackr.org)

Introduction

Pangenome graphs replace a single linear reference with a structure built from many haplotypes at once, so that mapping and variant calling no longer force every sample toward one arbitrary genome. The Human Pangenome Reference Consortium (HPRC) draft pangenome combines 47 phased, diploid assemblies with GRCh38 and CHM13 into graphs of this kind (Liao et al., 2023), and the PanGenome Graph Builder (PGGB) constructs them without designating any single input as the reference, aligning all haplotypes to one another and building a graph from the resulting alignment (Garrison et al., 2024). `vg giraffe` is the short-read mapper built for these graphs, using haplotype-constrained seed extension to map reads quickly against a large collection of paths threaded through a single sequence graph (Sirén et al., 2021).

Whether this combination scales to a full chromosome or genome is not settled. In their most recent description of Giraffe, Chang and colleagues state plainly that they have not been able to build genome-scale Giraffe indexes for PGGB graphs, and that PGGB's reference-free construction gives an unbiased view of homology at the cost of a topology that has so far resisted indexing (Chang et al., 2025). That statement is not accompanied by a quantitative, per-region measure of what “complex” means or by data on how often, and where, that complexity actually breaks the pipeline.

Other groups have approached the difficulty of pangenome regions from different angles. Li defines a set of sample-agnostic “easy” regions for short-read variant calling from hundreds of haplotype assemblies, using k-mer uniqueness rather than any property of a constructed graph (Li, 2025). Andreace and colleagues compare pangenome graphs built by different construction methods at the level of whole-graph and per-locus statistics (Andreace et al., 2023), and Dubois and colleagues define a pairwise edit distance between graphs built by different methods from the same genomes, which highlights local “hotspots” of disagreement, many of them coinciding with tandem repeats and other low-complexity sequence (Dubois et al., 2025). All of this work compares graphs to each other or scores regions from the input sequence directly; none of it asks whether a graph's own local topology, measured on the graph a mapper will actually index, predicts whether indexing and mapping succeed and how accurate the result is.

We address that question directly, at a scale small enough to test exhaustively: 100 kb windows on a single human chromosome. We ask whether a simple, explicit complexity score computed from local graph topology predicts (i) whether `vg autoindex` and `vg giraffe/vg surject` complete at all for a window, and (ii) how accurate the resulting short-read mapping is once they do.

Methods

Graph and reference data

We used the HPRC release 2 PGGB pangenome graph for chromosome 21 (parameters p98-k311), which contains GRCh38 and CHM13 as reference paths alongside 464 HPRC year-2 haplotypes (466 haplotype/reference paths in total) (Garrison et al., 2024; Liao et al., 2023). Chromosome 21 was chosen as the smallest human autosome, to keep chromosome-scale extraction and screening tractable on a single workstation. An independent GRCh38 chromosome 21 FASTA (UCSC hg38) was used for N-content screening and as a graph-independent coordinate reference. All graph operations (build, extract, stats, degree, viz) used the odgi toolkit (Guarracino et al., 2022). Full provenance, including checksums and download dates for every input file, is recorded alongside this repository.

Candidate window screening

Every non-overlapping 100 kb window on chromosome 21 was considered, after excluding the first and last 2% of the chromosome and any window with more than 5% N content in the GRCh38 reference, leaving 367 candidate windows. Each was extracted from the graph with `odgi extract` and summarized with `odgi stats` and `odgi degree`. Windows were dropped from further analysis under two criteria. First, a degenerate-window filter (`edge_count == 0`, or fewer than 400 raw path fragments, or an extracted size more than three times the requested window) removed 22 windows clustered in the pericentromeric and acrocentric part of the chromosome, leaving 345. Second, because the raw path-fragment count used above does not distinguish a haplotype that is finely fragmented from a haplotype that is simply absent, we additionally counted the number of *distinct* haplotypes actually threading each window (parsed from the PanSN `sample#hap` path-name prefix via `odgi paths -L`). Nineteen windows, all within a contiguous stretch immediately flanking the centromere, had fewer than 450 of the 466 possible haplotypes represented and were removed, leaving 326 windows that were carried forward for scoring and sampling.

Complexity score

For each of the 326 valid windows we computed four graph-topology statistics from `odgi stats/odgi degree`: node density (nodes per kilobase of reference sequence), the edge-to-node ratio, sequence inflation (total graph sequence relative to the reference interval), and maximum node degree. Each statistic was log10-transformed and z-scored across the 326-window population, and the four z-scores were averaged into a single, unitless local complexity score. This score is intentionally a description of graph shape rather than of any biological property of the underlying sequence; how well it tracks biologically meaningful difficulty is exactly the question this report addresses.

Known-hazard annotation

Complexity score alone cannot see sequence similarity to other parts of the genome, since it is computed from one local subgraph. To capture that separately, every valid window was checked against two UCSC tracks for chromosome 21, `genomicSuperDups` (Bailey et al., 2001) and `simpleRepeat` (Tandem Repeats Finder, (Benson, 1999)), both retrieved once for the whole chromosome and intersected locally rather than queried per window (Kent et al., 2002). A binary hazard flag was set for a window if it overlapped a segmental duplication at 90% identity or higher, or a tandem-repeat array spanning 1.4 kb or more; these thresholds were chosen because they correctly flagged every problem window identified during preliminary, hand-picked testing of this pipeline (a genuine inter-chromosomal segmental duplication and three large tandem-repeat/NUMT-containing windows). Under this definition, 77 of the 326

windows (23.6%) carry a hazard flag. Crossing the hazard flag with a complexity tier (above or below the population median complexity score) gives a 2x2 design with four strata.

Region sampling and pipeline

Fifty windows were rank-sampled within each of the four complexity-tier-by-hazard-status strata, 25 per tier and 25 per hazard status by construction, so that rare combinations such as a low-complexity window carrying a known hazard would not be missed by chance. Each of the 50 regions was carried through an identical pipeline. Sequence graphs were extracted with `odgi extract` at the default subpath-merging distance; a minority of extractions that expanded past three times the requested window size, a known behaviour of the default merge distance next to large collapsed repeats, were re-extracted with merging disabled. Giraffe indexes were built with `vg autoindex --workflow sr-giraffe` (Sirén et al., 2021). For each region we simulated 30,000 error-free paired-end 150 bp reads (fragment length $350 \text{ bp} \pm 35 \text{ bp}$) from the GRCh38 interval with `wgsim` (Li, 2011), with a fixed seed for reproducibility. Reads were mapped with `vg giraffe` and surjected onto the linear GRCh38 path with `vg surject` (Sirén et al., 2021); BAM records were inspected with `samtools` (Danecek et al., 2021). A mapped read was scored correct if its absolute position, converted back from the region's local subpath coordinate, fell within 10 bp of either candidate fragment-boundary position implied by the simulated fragment (the read simulator does not record simulated strand, so either boundary is an acceptable match). Every pipeline stage for a region — extraction, indexing, simulation, mapping, surjection, evaluation — was run under a 30-minute wall-clock budget on a single machine (Apple Silicon, 16 cores, 128 GB memory); a region that had not produced a result within that budget was recorded as a pipeline failure. Two genes flagged as biologically notable in this analysis (the ribosomal RNA gene *RNA5-8SN3* and the *MIR6724* microRNA cluster overlapping the single most complex window screened, which sits outside the final 326-window pool) were identified by looking up their coordinates against Ensembl gene annotation (Dyer et al., 2025). All downstream tabulation and the figures in this report were produced with `pandas` (McKinney, 2010) and `matplotlib` (Hunter, 2007).

Results

Of the 50 sampled regions, 46 completed the full pipeline and 4 did not finish within the 30-minute budget. All four incomplete regions fell in the above-median complexity tier: 0 of 25 low-complexity regions failed to complete, against 4 of 25 high-complexity regions (16%). Splitting instead by hazard status gives a much weaker separation on this same outcome — 1 of 25 hazard-free regions failed, against 3 of 25 hazard-flagged regions (4% versus 12%) — so complexity tier, not hazard status, is what tracks whether the pipeline finishes at all.

Mapping accuracy among the 46 regions that did complete shows the opposite pattern. Split by complexity tier, minimum accuracy was 98.7% in the low tier and 99.3% in the high tier — complexity tier does not identify the worst-mapping region. Split by hazard status, the separation is clean: mean accuracy was 99.92% without a hazard flag and 99.84% with one, and minimum accuracy was 99.53% without a flag against 98.69% with one. The single worst-accuracy region in the entire sample, at 98.69%, is also the single lowest-complexity region in the whole 326-window population (complexity score -2.17). It carries a hazard flag: `genomicSuperDups` lists 41 segmental-duplication entries overlapping this window at 90-97% identity, matching paralogous sequence on six other chromosomes. A complexity score built entirely from one local subgraph has no way to see that this locus is, in effect, duplicated elsewhere in the genome, and so it scores this window as the simplest in the whole population.

Figure 1 shows why a stratified rather than a uniform sample was necessary in the first place: complexity score is heavily right-skewed across the 326-window population, with 77.6% of windows sitting within half a standard deviation of the median. A sample drawn uniformly, or even one rank-sampled across the score's full range without regard to hazard status, would

rarely land on the rare, extreme, or hazard-flagged windows that turn out to carry the interesting outcomes.

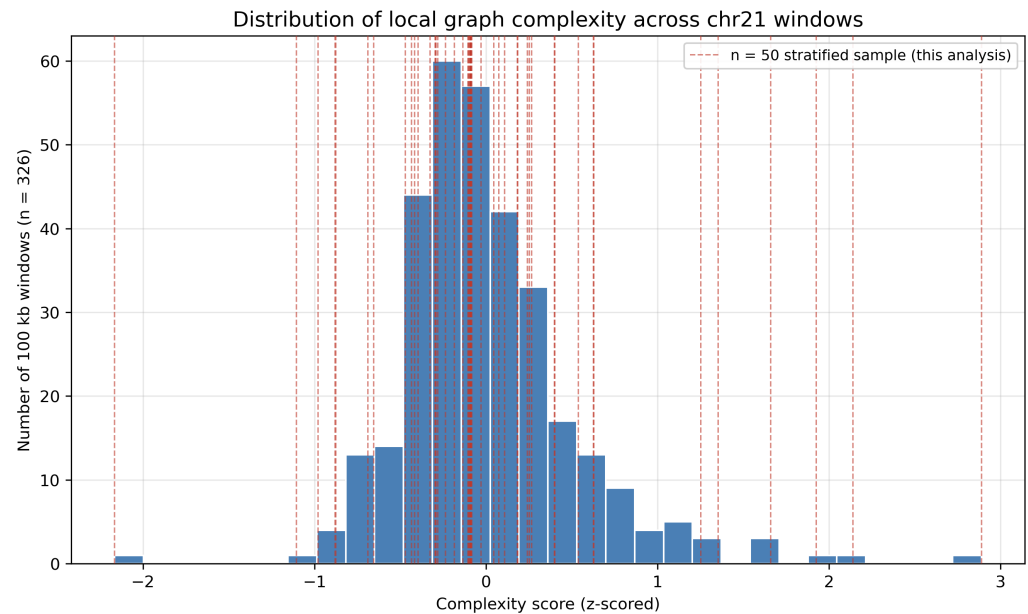


Figure 1: Distribution of local complexity score across the 326 valid 100 kb windows on chromosome 21, with the 50 sampled regions marked by dashed lines

Figure 2 summarizes the full result. Panel a plots complexity score against mapping accuracy for all 50 regions, colored by hazard status, with the four incomplete regions marked at the bottom of the axis rather than omitted; panel b shows the failure-rate split by complexity tier; panel c shows mean and minimum accuracy split by hazard status. Read together, the three panels make the same point three different ways: complexity score and hazard status are each doing real, and different, work.



Figure 2: Complexity score, pipeline completion, and mapping accuracy across the 50 sampled regions. (a) complexity score versus accuracy, colored by known-hazard status, with pipeline failures marked at the bottom of the axis; (b) pipeline failure rate by complexity tier; (c) mean and minimum accuracy among completed regions, by hazard status

Two qualifications matter for how these numbers should be read. First, “did not complete” here means no result within a fixed 30-minute, single-machine budget, not a proof that a region is impossible to index or map. Two regions encountered during preliminary testing of this pipeline, evaluated under a shorter 10-minute budget at the time, looked like failures and then completed successfully once given 30 minutes; one of them is in fact part of this sample,

the single highest-complexity region tested (complexity score 2.89), and it completed here in about 26 minutes at 99.78% accuracy. The 16% failure rate among high-complexity regions is therefore better read as an upper bound under this specific compute budget than as a fixed property of the graph. Second, that 16% rests on four failure events out of 25 regions, and the hazard-versus-failure comparison on even fewer; the direction of both effects is clear, but the exact percentages carry real sampling uncertainty and should not be treated as precise population parameters.

Discussion

Two properties of a 100 kb pangenome window, measured independently of one another, predict two different things. A complexity score built purely from local graph topology — node density, branching, and how much the graph's sequence content is inflated relative to the reference — predicts whether the indexing and mapping pipeline completes at all within a fixed compute budget. It does not predict how accurate the mapping is once the pipeline does complete. A separate annotation for known problematic sequence classes, segmental duplications and large tandem repeats, drawn from two long-established UCSC tracks, predicts accuracy degradation among completed regions cleanly, and does not predict whether the pipeline completes. Neither measure substitutes for the other, and a screen that used only one of them would miss real, quantifiable risk of the kind the other one catches.

This result sits alongside Chang and colleagues' report that Giraffe indexing has not yet been made to work at genome scale for PGGB graphs (Chang et al., 2025). Our result does not resolve that problem; it probes a smaller version of it directly. At the 100 kb scale we tested, indexing and mapping are tractable most of the time, and the cases where they are not are not confined to the single most extreme window on the chromosome. Instead, a meaningful minority of above-median-complexity windows fail outright under an ordinary compute budget, in a way a genome-wide extension of this pipeline would need to budget for as an outcome in its own right rather than an occasional, ignorable exception.

The most direct, practical recommendation this analysis supports is that a genome-wide screen built on this approach should not rely on a topology-only complexity score as its only filter. Checking candidate regions against existing, cheaply computed sequence-hazard annotation before trusting a complexity ranking would have caught the worst accuracy result in our own sample, which a complexity score alone positively misclassified as the simplest region tested. A generous, resource-aware per-region timeout matters for the same reason: at least two regions in this project that first looked like hard failures under a short timeout turned out to be merely slow, and a screen that gives up too quickly will overstate its own failure rate.

Several limitations bound how far these results generalize. Reads were simulated error-free from the linear reference, so this measures graph and mapper behaviour under idealized input, not real Illumina sequencing. This analysis covers a single chromosome, the smallest human autosome, and we did not test whether the specific failure and hazard rates reported here hold on chromosomes with a different balance of segmental duplication, tandem repeat, and acrocentric or pericentromeric content. All compute ran on a single workstation with a fixed timeout, which sets a concrete but arbitrary ceiling on what counts as "failure" in the results above. And a truth model based on a single simulated coordinate per read cannot distinguish a genuine mapping error from a read placed, correctly, at one of several biologically valid locations in a duplicated region — exactly the situation a hazard-flagged window is most likely to create. Extending this analysis to more chromosomes, real sequencing reads, and a repeat-aware evaluation of mapping correctness would each address one of these limitations directly.

Acknowledgements

We thank the organizers and participants of BioHackathon 2026 for the time and computing environment that made this analysis possible, and the HPRC, PGGB, and vg development teams for making the underlying graphs and tools openly available.

References

Author contributions

Kazumichi Fujiwara: Conceptualization, Investigation, Software, Writing – original draft; Yosuke Kawai: Writing – review & editing.

References

- Andreace, F., Lechat, P., Dufresne, Y., & Chikhi, R. (2023). Comparing methods for constructing and representing human pangenome graphs. *Genome Biology*, 24(1), 274. <https://doi.org/10.1186/s13059-023-03098-2>
- Bailey, J. A., Yavor, A. M., Massa, H. F., Trask, B. J., & Eichler, E. E. (2001). Segmental duplications: organization and impact within the current human genome project assembly. *Genome Research*, 11(6), 1005–1017. <https://doi.org/10.1101/gr.187101>
- Benson, G. (1999). Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Research*, 27(2), 573–580. <https://doi.org/10.1093/nar/27.2.573>
- Chang, X., Novak, A. M., Eizenga, J. M., Sirén, J., Monlong, J., Negi, S., Andreace, F., Nag, S., Kyriakidis, K., Hickey, G., Hwang, S., Délot, E. C., Carroll, A., Shafin, K., Chang, P.-C., Okamoto, F., & Paten, B. (2025). Rapid, accurate long- and short-read mapping to large pangenome graphs with vg Giraffe. *bioRxiv*. <https://doi.org/10.1101/2025.09.29.678807>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008. <https://doi.org/10.1093/gigascience/giab008>
- Dubois, S., Zytynicki, M., Lemaitre, C., & Faraut, T. (2025). Pairwise graph edit distance characterizes the impact of the construction method on pangenome graphs. *Bioinformatics*, 41(6), btaf291. <https://doi.org/10.1093/bioinformatics/btaf291>
- Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1), D948–D957. <https://doi.org/10.1093/nar/gkae1071>
- Garrison, E., Guarracino, A., Heumos, S., Villani, F., Bao, Z., Tattini, L., Hagmann, J., Vorbrugg, S., Marco-Sola, S., Kubica, C., Ashbrook, D. G., Hickey, G., Sirén, J., Eizenga, J. M., Consortium, H. P. R., Cartwright, R. A., Barrio, A. M., Ghurye, J., Abramson, A., et al. (2024). Building pangenome graphs. *Nature Methods*, 21, 2008–2012. <https://doi.org/10.1038/s41592-024-02430-3>
- Guarracino, A., Heumos, S., Nahnsen, S., Prins, P., & Garrison, E. (2022). ODGI: understanding pangenome graphs. *Bioinformatics*, 38(13), 3319–3326. <https://doi.org/10.1093/bioinformatics/btac308>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kent, W. J., Sugnet, C. W., Furey, T. S., Roskin, K. M., Pringle, T. H., Zahler, A. M., & Haussler, D. (2002). The human genome browser at UCSC. *Genome Research*, 12(6), 996–1006. <https://doi.org/10.1101/gr.229102>
- Li, H. (2011). wgsim: a small tool for simulating sequence reads from a reference genome.
- Li, H. (2025). Finding easy regions for short-read variant calling from pangenome data. *GigaScience*, 14, giab103. <https://doi.org/10.1093/gigascience/giab103>
- Liao, W.-W., Asri, M., Ebler, J., Doerr, D., Haukness, M., Hickey, G., Lu, S., Lucas, J. K., Monlong, J., Abel, H. J., et al. (2023). A draft human pangenome reference. *Nature*, 617(7960), 312–324. <https://doi.org/10.1038/s41586-023-05896-x>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>

Sirén, J., Monlong, J., Chang, X., Novak, A. M., Eizenga, J. M., Markello, C., Sibbesen, J. A., Hickey, G., Chang, P.-C., Carroll, A., Gupta, N., Gabriel, S., Blackwell, T. W., Ratan, A., Taylor, K. D., Rich, S. S., Rotter, J. I., Haussler, D., Garrison, E., & Paten, B. (2021). Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science*, 374(6574), abg8871. <https://doi.org/10.1126/science.abg8871>



Appendices

Full methodology detail, per-region result tables, and the narrative logs documenting the preliminary hand-picked testing referenced above are available in this repository's `README.md` and `logs/` directory.